

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

Институт компьютерных наук и технологического образования
Кафедра компьютерных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения
и обработки больших данных»
форма обучения – очная

Курсовая работа

по дисциплине «Прикладные информационные технологии»
Развертывание микросервисной архитектуры Telegram-бота и веб-лендинга с
использованием Docker Compose и автоматизацией выпуска сертификатов Let's
Encrypt

Обучающейся 3 курса
Мельник Н. О.

Руководитель:
старший преподаватель Аксютин П. А.

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
Глава 1. Микросервисная архитектура.....	5
1.1. Понятие микросервисной архитектуры	5
1.2. Контейнеризация и Docker	9
1.3. Сертификаты Let's Encrypt.....	12
Глава 2. Реализация задачи с применением компьютерных технологий ...	13
2.1. Программная реализация Telegram-бота	13
2.2. Программная реализация веб-лендинга	14
2.3. Сравнение технологий и сервисов	15
ЗАКЛЮЧЕНИЕ	18
СПИСОК ЛИТЕРАТУРЫ	20
ПРИЛОЖЕНИЕ А.....	22
ПРИЛОЖЕНИЕ Б	25

ВВЕДЕНИЕ

Актуальность темы курсовой работы заключается в том, что современные программные системы всё чаще строятся по принципу микросервисной архитектуры, позволяющей разделить сложное приложение на независимые, слабо связанные компоненты. Такой подход обеспечивает гибкость разработки, независимость развёртывания, масштабируемость и отказоустойчивость. Telegram-боты стали неотъемлемой частью коммуникации между компаниями и пользователями, а веб-лендинги служат эффективным инструментом для презентации и привлечения аудитории. Интеграция этих двух сервисов в единую микросервисную архитектуру с использованием современных инструментов контейнеризации и автоматизации является актуальной задачей, особенно в контексте импортозамещения и необходимости развёртывания отечественных сервисов на российских серверах.

Практическая значимость развёртывания микросервисной архитектуры заключается в демонстрации полноценного цикла разработки и развёртывания микросервисной архитектуры. Результаты работы могут быть использованы в качестве шаблона для развёртывания аналогичных проектов, а также для изучения современных методов контейнеризации и автоматизации в курсах по разработке программного обеспечения.

Целью курсовой работы является развёртывание микросервисной архитектуры Telegram-бота и веб-лендинга с использованием Docker Compose и автоматизацией выпуска сертификатов Let's Encrypt.

На основе цели возникли задачи:

1. Сбор информации и материалов о микросервисной архитектуре
2. Сбор информации и материалов о контейнеризации и docker
3. Сбор информации и материалов о сертификатах Let's Encrypt
4. Анализ и развёртывание с помощью применения информационных технологий микросервисной архитектуры Telegram-бота и веб-лендинга

Предметом вычислительного эксперимента являются процессы контейнеризации, оркестрации и автоматизации выпуска SSL-сертификатов при развёртывании микросервисной архитектуры.

Объектом вычислительного эксперимента является микросервисная архитектура, включающая Telegram-бота и веб-лендинг компании «Галактика» (производитель молочной продукции).

Краткое описание структуры работы: Работа состоит из двух частей – теоретической, где рассматриваются основные понятия микросервисной архитектуры, контейнеризации и docker, автоматизация выпуска сертификатов Let's Encrypt. Вторая часть состоит в реализации и анализе развёртывания микросервисной архитектуры Telegram-бота и веб-лендинга с помощью применения информационных технологий.

Глава 1. Микросервисная архитектура

1.1. Понятие микросервисной архитектуры

Микросервисная архитектура — это способ создания программных продуктов, предполагающий разработку независимых друг от друга модулей. Каждая часть приложения отвечает за определенную задачу и может быть изменена или расширена без дополнений в других. При этом сервисы взаимодействуют между собой с помощью обмена сообщениями.

Основным принципом данного подхода является декомпозиция приложения на набор небольших, слабо связанных и независимо развертываемых сервисов, каждый из которых отвечает за выполнение определенной бизнес-функции. Такая структура обеспечивает высокую степень масштабируемости системы, позволяя увеличивать производительность отдельных компонентов без необходимости масштабирования всего приложения целиком. Кроме того, микросервисная архитектура способствует повышению гибкости разработки, предоставляя возможность использовать различные технологии и языки программирования для реализации отдельных сервисов, что позволяет выбирать наиболее подходящие инструменты для решения конкретных задач. Независимость сервисов также обеспечивает улучшенную отказоустойчивость системы, так как сбой в работе одного компонента не приводит к полному отказу всего приложения [1].

Основные принципы проектирования микросервисов:

1. База данных для каждого сервиса.

База данных для каждого сервиса означает, что каждый микросервис должен иметь собственное хранилище данных, изолированное от других сервисов. Данный подход позволяет различным сервисам использовать разные типы баз данных, оптимально подходящие для их конкретных задач.

Улучшается масштабируемость, так как независимые базы данных могут быть масштабированы отдельно, в соответствии с нагрузкой на конкретный

сервис. Наконец, достигается изоляция отказов, поскольку проблемы с базой данных одного сервиса не влияют на работоспособность других сервисов. Следует отметить, что данный принцип также создает определенные вызовы, такие как необходимость обеспечения согласованности данных между сервисами и усложнение процесса выполнения запросов, затрагивающих данные из нескольких сервисов.

2. Принцип слабой связанности.

Связанность представляет собой степень взаимосвязи между сервисами. При создании систем необходимо стремиться к максимальной независимости сервисов, то есть их связанность должна быть минимальной [2].

Основные аспекты данного принципа включают независимость развертывания и автономность, при которой каждый сервис может быть развернут, обновлен или заменен без влияния на другие сервисы. А коммуникация между сервисами осуществляется через четко определенные API. Реализация принципа слабой связанности способствует повышению гибкости системы.

Типы связанности от слабой к сильной: предметная, сквозная, общая, по содержимому.

Предметная связанность неизбежна при построении микросервисной архитектуры, она отражает обращение одного микросервиса к функциональности другого.

Сквозная связанность проявляется в последовательности вызовов между сервисами. Для выполнения определенной задачи, первый микросервис обращается ко второму сервису через третий, в этом случае третий выполняет только функцию передачи информации дальше.

Общая связанность относится к зависимостям, возникающим из-за использования общих ресурсов или компонентов. Это может быть общая библиотека аутентификации, база данных клиентов или API-шлюз. Хотя такая связанность может упростить разработку, она может усложнить независимое развертывание и обновление сервисов. К примеру, два микросервиса меняют в

одной базе данных статус заказов, обработчик заказов выставляет статусы размещен, оплачен и завершен, а микросервис склад собирается, оплачен.

Связанность по содержимому является наиболее проблематичной. Она возникает, когда микросервисы зависят от внутренней реализации друг друга, один микросервис меняет базу данных другого в обход функциональной логики этого микросервиса [3].

3. Принцип единственной ответственности.

В рамках микросервисов этот принцип постулирует, что каждый сервис должен быть ответственен за одну, четко определенную бизнес-функцию или область. Границы ответственности сервиса должны быть четко очерчены и соответствовать определенному контексту предметной области. Применение данного принципа приводит к упрощению разработки и поддержки, так как каждый сервис фокусируется на решении конкретной задачи, что облегчает понимание и модификацию кода. Наконец, повышается надежность системы, так как изоляция функциональности снижает риск каскадных отказов [4].

Основные критерии сравнения монолитной и микросервисной архитектуры представлены в таблице 1 [5].

Таблица 1. Основные критерии сравнения монолитной и микросервисной архитектуры

Критерий	Микросервисная архитектура	Монолитная архитектура
Структура системы	Система разделена на независимые компоненты (микросервисы), каждый из которых решает отдельную бизнес-задачу, имеет собственную логику и базу данных.	Все компоненты тесно связаны и работают как единое целое. Вся логика приложения, включая бизнес-процессы, механизмы взаимодействия с базами данных, пользовательский интерфейс и API, интегрированы в рамках одного приложения.

Скорость релизов	Чтобы запустить новые функции или обновить существующие, достаточно изменить один модуль. Это ускоряет разработку и позволяет чаще выпускать обновления.	Чтобы протестировать релиз и подготовить к нему приложение, нужно обновить всю систему. Это может привести к неожиданным сбоям и увеличить время отладки.
Диапазон технологий	Для каждого сервиса можно выбрать свой язык программирования, способ хранения данных и необходимые библиотеки.	Код монолита — единое целое, поэтому вся команда должна придерживаться уже выбранных инструментов и методов.
Масштабируемость	Поддерживает горизонтальное масштабирование — можно увеличивать производительность за счёт запуска дополнительных копий только нужных сервисов.	Масштабирование требует копирования всего приложения, даже если нужно повысить производительность только одной его части.
Устойчивость к сбоям	Сбой одного сервиса не приводит к отказу всего приложения.	Все элементы монолита связаны друг с другом напрямую или косвенно: сбой внутри одного модуля может вызвать полный отказ системы.
Управление	Координация между множеством сервисов требует дополнительных усилий и инструментов.	Процессы разработки, внедрения и масштабирования осуществляются для всей системы как единого целого, без разделения на отдельные компоненты.
Согласованность данных	Могут возникать сложности с поддержанием целостности и актуальности информации в распределённой архитектуре.	У всех компонентов есть прямой доступ к одной и той же базе данных, что позволяет беспрепятственно обмениваться данными.

Требования к инфраструктуре и DevOps	Необходимы системы мониторинга, логирования, трассировки, инструменты проверки связанности.	Монолит можно развернуть на одной физической машине.
--------------------------------------	---	--

1.2. Контейнеризация и Docker

Контейнеризация — это способ упаковки приложения и всех его зависимостей в один образ, который запускается в изолированной среде, не влияющей на основную операционную систему [6].

Каждый микросервис может быть упакован в отдельный контейнер. Это позволяет:

- Обеспечить независимость сервисов;
- Упростить масштабирование отдельных компонентов;
- Быстро обновлять и развёртывать сервисы без влияния на другие.

Контейнеризация напоминает виртуализацию, но технологии отличаются друг от друга. Виртуализация работает как отдельный компьютер со своей операционной системой и виртуальным оборудованием. Внутри одной операционной системы можно запустить другую. При контейнеризации виртуальная среда может запускаться прямо из ядра основной ОС и не виртуализирует оборудование. Поэтому контейнер может работать только в той же операционной системе, что и основная. Так как контейнеры не виртуализируют оборудование, они потребляют меньше ресурсов [7].

Docker - это платформа с открытым исходным кодом, которая облегчает разработку, развертывание и управление приложениями за счет контейнеризации. Она позволяет разработчикам упаковывать приложения и их зависимости в легкие переносимые контейнеры, обеспечивая стабильную работу приложений в различных средах и на разных этапах разработки.

Docker контейнер — это стандартизированный, изолированный и портативный пакет программного обеспечения, который включает в себя все

необходимое для запуска приложения, включая код, среду выполнения, системные инструменты, библиотеки и настройки. Контейнеры позволяют упаковать приложение и все его зависимости в единый объект, который может быть запущен на любой системе, поддерживающей Docker, без изменения среды выполнения.

Docker работает с компонентами:

- **Dockerfile.** Это файл для предварительной работы, набор инструкций, который нужен для записи образа. В нем описывается, что должно находиться в образе, какие команды, зависимости и процессы он будет содержать.

Для сборки образа из Dockerfile используют команду:

```
docker build <докерфайл>
```

- **Docker Image.** Это образ — неизменяемый файл, из которого разворачивается контейнер.

Для этого нужно запустить образ в клиенте с помощью специальной команды:

```
docker run <образ>
```

- **Docker Registry.** Это реестр, или репозиторий — открытая или закрытая база образов.

К ней можно подключиться через клиент Docker и загрузить нужный с помощью команды:

```
docker pull <образ>
```

При запуске команды *docker run* программа сначала проверяет, есть ли нужный образ в локальном хранилище. Если его нет, она сама находит файл в репозитории и скачивает на компьютер.

- **Docker Container.** Это уже готовый и развернутый контейнер, который находится на каком-либо устройстве [8].

Шаги по созданию Docker-ориентированного микросервисного приложения:

1. Проектирование микросервисов: Разбейте приложение на несколько небольших модульных сервисов, которые можно независимо разрабатывать, развертывать и масштабировать. Каждый микросервис должен иметь четко определенную ответственность и взаимодействовать с другими через API или очереди обмена сообщениями.
2. Создайте Docker-файлы: Для каждого микросервиса создайте *Dockerfile*, в котором укажите базовый образ, код приложения, зависимости и конфигурацию, необходимые для создания образа Docker. Этот образ используется для развертывания микросервисов в виде контейнеров.
3. Сборка образов Docker: Выполните команду *Docker build* для создания образов Docker для каждого микросервиса, следуя инструкциям, указанным в соответствующих Dockerфайлах.
4. Создать сеть: Создайте сеть между контейнерами, чтобы обеспечить взаимодействие между микросервисами. Для создания сети можно использовать *Docker Compose* или инструмент оркестровки контейнеров.
5. Настроить балансировку нагрузки: Настройте балансировщик нагрузки для распределения запросов между экземплярами микросервисов, обеспечивая оптимальную производительность и отказоустойчивость.
6. Развертывание микросервисов: Разверните микросервисы в виде контейнеров Docker с помощью выбранной вами платформы оркестровки контейнеров. Это позволит создать среду, в которой микросервисы смогут работать, взаимодействовать друг с другом и масштабироваться по требованию [9].

Docker Compose: множественные образы

Считается, что один контейнер должен выполнять одну функцию. Поэтому для настройки сложной среды понадобится несколько образов.

ПО позволяет управлять несколькими образами и контейнерами как одним. С помощью *Docker Compose* можно создать файл *YAML* для определения служб и с одной командой запускать и останавливать все что нужно. Инструмент

способен ускорить разработку: в многоконтейнерных приложениях количество контейнеров может достигать до нескольких десятков [10].

1.3. Сертификаты Let's Encrypt

Сертификаты Let's Encrypt — это бесплатные SSL/TLS-сертификаты, которые используются для обеспечения защищённого соединения (HTTPS) в микросервисных архитектурах. Они позволяют шифровать данные, передаваемые между клиентом и сервисом, и повышают безопасность взаимодействия [11].

Процесс получения сертификата включает несколько этапов:

1. Подтверждение владения доменом. Let's Encrypt проверяет, что пользователь контролирует домен, для которого запрашивается сертификат. Это может быть сделано через HTTP-01 (размещение определённого HTTP-кода) или DNS-01 (установка определённых DNS-записей).
2. Выпуск сертификата. После успешной проверки Let's Encrypt выпускает сертификат.
3. Автоматическое обновление. Сертификаты автоматически продлеваются по расписанию [12].

Для автоматизации выпуска сертификатов Let's Encrypt в микросервисных средах часто используется cert-manager — проект для Kubernetes, который позволяет управлять сертификатами.

Некоторые возможности:

- Автоматическое предоставление сертификатов. cert-manager может запрашивать сертификаты у Let's Encrypt и управлять их выпуском и обновлением;
- Поддержка различных методов проверки;
- Интеграция с другими инструментами [13].

Глава 2. Реализация задачи с применением компьютерных технологий

2.1. Программная реализация Telegram-бота

Необходимость в написании кода Telegram-бота связана с тем, что он обеспечивает интерактивное взаимодействие с пользователями компании «Галактика». Бот предоставляет информацию о компании, производстве, брендах, новостях и контактах, позволяя пользователям быстро получать актуальные данные без необходимости посещать сайт. Язык программирования Python выбран из-за наличия удобной библиотеки `python-telegram-bot` и кросс-платформенности. Кроме того, Python имеет богатую экосистему для парсинга данных (`BeautifulSoup4`, `lxml`), что позволило автоматически загружать информацию с сайта компании.

Структура проекта:

- `bot.py` # основной файл запуска
- `commands.py` # обработчик команд
- `constants.py` # константы
- `keyboards.py` # клавиатуры для меню
- `templates.py` # форматирование текста
- `requirements.txt` # зависимости Python
- `parsing/` # данные парсинга сайта
- `utils/` # вспомогательные функции

Анализ работы программы: При запуске бот загружает данные из JSON-файла `parsing/bot_data.json`, который был получен парсингом сайта компании. Основная функция `handle_deep_link` обрабатывает глубокие ссылки вида `https://t.me/bot?start=about`, которые передаются из веб-лендинга. В зависимости от переданного параметра бот отправляет пользователю соответствующую информацию. Все команды логируются для отладки.

С результатами работы можно ознакомиться в приложении. (см. ПРИЛОЖЕНИЕ А)

2.2. Программная реализация веб-лендинга

Необходимость в создании веб-лендинга связана с тем, что он служит презентационной страницей для Telegram-бота. Лендинг содержит 6 информационных карточек, соответствующих разделам бота, и позволяет пользователям быстро перейти в Telegram с предзаполненной командой. Язык разметки HTML и каскадные таблицы стилей CSS выбраны из-за простоты создания статических сайтов, минимальных требований к ресурсам и возможности запуска в любом браузере.

Для развёртывания веб-лендинга на сервере используется технология контейнеризации Docker. Лендинг упаковывается в Docker-образ на основе веб-сервера NGINX в версии `alpine`. Это позволяет:

- Запускать лендинг изолированно от других сервисов;
- Легко переносить и развёртывать на любом сервере с поддержкой Docker;
- Масштабировать независимо от других компонентов системы.

Для обеспечения защищённого соединения (HTTPS) на сервере используется связка nginx-проху и acme-companion, которая автоматически получает и обновляет SSL-сертификаты Let's Encrypt.

Развёртывание на сервере

Процесс развёртывания веб-лендинга на сервере включает следующие шаги:

- Установка Docker и Docker Compose на сервере;
- Настройка DNS — домен должен указывать на IP сервера;
- Копирование файлов проекта на сервер с помощью scp;
- Запуск контейнеров командой `docker-compose up -d`.

Анализ работы программы: Веб-лендинг представляет собой статическую страницу, которая не требует серверной логики. Каждая карточка обернута в тег `<a>`, что делает её кликабельной. При нажатии на карточку открывается

Telegram-бот с командой, соответствующей разделу (например, https://t.me/galaxy_milk_bot?start=about). Внизу страницы расположены две дополнительные кнопки: «Контакты» (ведёт в бот с командой `contacts_inline`) и «Веб-сайт компании» (ведёт на официальный сайт). Основная кнопка «Открыть бота в Telegram» открывает чат с ботом без дополнительной команды.

С результатами работы можно ознакомиться в приложении. (см. ПРИЛОЖЕНИЕ Б)

2.3. Сравнение технологий и сервисов

В ходе разработки микросервисной архитектуры потребовалось выбрать инструменты для реализации следующих ключевых задач: контейнеризация, reverse-proxy, автоматизация SSL-сертификатов. Анализ альтернативных технологий и обоснование выбранных решений представлены в таблице 2.

Таблица 2. Выбор reverse-proxy

Критерий	nginx-proxy	Traefik	Caddy	Ручной NGINX
Автоматический HTTPS	Есть (acme-companion)	Есть	Есть	Нет
Авто-конфигурация	Есть (Docker socket)	Есть	Нет	Нет
Сложность настройки	Низкая	Средняя	Низкая	Высокая
Документация	Хорошая	Отличная	Средняя	Отличная
Интеграция с Docker	Есть	Есть	Нет	Нет

Выбран nginx-proxy:

- Простая настройка через переменные окружения `VIRTUAL_HOST`, `LETSENCRYPT_HOST`;
- Родная интеграция с `acme-companion` для автоматического HTTPS;
- Основан на NGINX — проверенном и производительном веб-сервере;
- Не требует написания конфигурационных файлов вручную.

Выбран не ручной NGINX: При добавлении новых сервисов потребовалось бы вручную редактировать конфиг и перезагружать сервер, что противоречит принципам автоматизации.

Выбор инструмента автоматизации HTTPS представлен в таблице 3.

Таблица 3. Выбор инструмента автоматизации HTTPS

Критерий	acme-companion	Certbot	Traefik
Автоматическое получение	Есть	Есть	Есть
Автоматическое обновление	Есть	Нет (требуется cron/systemd)	Есть
Интеграция с Docker	Есть (через socket)	Нет (ручная)	Есть
Требует доступа к shell	Нет	Есть	Нет
Дополнительная настройка	Минимальная	Средняя	Средняя

Выбран acme-companion (совместно с nginx-proxy):

- Полная автоматизация: сертификаты получаются и обновляются без участия человека;
- Работает через Docker socket, отслеживая новые контейнеры с LETSENCRYPT_HOST;
- Не требует настройки cron-заданий или systemd-таймеров;
- Интегрируется с nginx-proxy «из коробки».

Итоговое сравнение выбранных решений представлена в таблице 4.

Таблица 4. Итоговое сравнение выбранных решений

Компонент	Выбранное решение	Альтернатива	Причина выбора
Контейнеризация	Docker	Podman, LXC	Популярность, Docker Compose
Reverse-proxy	nginx-proxy	Traefik, Caddy	Простота, интеграция с acme-companion

Автоматизация HTTPS	acme-companion	Certbot	Полная автоматизация
------------------------	----------------	---------	----------------------

Вывод по результатам сравнения: Выбранные технологии обеспечивают оптимальный баланс между функциональностью, сложностью реализации и затратами на поддержку. Docker + nginx-проxy + acme-companion образуют связку, которая минимизирует ручную настройку и автоматизирует получение HTTPS. Python с библиотекой python-telegram-bot позволяет быстро разрабатывать и поддерживать бота.

ЗАКЛЮЧЕНИЕ

По результатам выполненной работы можно сделать выводы:

1. **Микросервисная архитектура** — это способ построения программных систем, при котором приложение разделяется на набор небольших, независимых и слабо связанных сервисов. Каждый сервис отвечает за выполнение конкретной бизнес-функции, имеет собственную базу данных и может быть разработан, развёрнут и масштабирован независимо от других сервисов. Основные принципы микросервисной архитектуры включают: база данных для каждого сервиса, принцип слабой связанности и принцип единственной ответственности. В сравнении с монолитной архитектурой микросервисный подход обеспечивает более высокую скорость релизов, широкий диапазон используемых технологий, гибкое масштабирование и устойчивость к сбоям, однако требует дополнительных усилий для управления распределённой системой и поддержания согласованности данных.

2. **Контейнеризация и Docker** являются ключевыми технологиями для реализации микросервисной архитектуры. Контейнеризация позволяет упаковать приложение и все его зависимости в изолированный образ, который можно запустить на любой системе с поддержкой Docker. В отличие от виртуальных машин, контейнеры легковесны, запускаются за секунды вместо минут и потребляют меньше ресурсов. Docker предоставляет следующие компоненты: Dockerfile (инструкции для сборки образа), Docker Image (неизменяемый файл образа), Docker Registry (хранилище образов), Docker Container (запущенный экземпляр образа). Docker Compose позволяет оркестрировать многоконтейрные приложения с помощью одной команды (`docker-compose up -d`), что особенно удобно при работе с несколькими микросервисами.

3. **Let's Encrypt** — это бесплатные SSL/TLS-сертификаты, которые используются для обеспечения защищённого HTTPS-соединения. Процесс получения сертификата включает подтверждение владения доменом, выпуск

сертификата и его автоматическое обновление. Срок действия сертификатов Let's Encrypt составляет 90 дней, что стимулирует автоматизацию процесса обновления. Для работы с сертификатами существуют различные инструменты: Certbot (официальный клиент), acme-companion (интеграция с Docker) и встроенные решения в Traefik и Caddy. В работе использована связка nginx-proxu и acme-companion, которая автоматически отслеживает новые контейнеры, получает сертификаты через HTTP-01 верификацию и обновляет их каждые 12 часов без участия человека.

4. **Практическая реализация** включала разработку Telegram-бота с поддержкой глубоких ссылок (функция `handle_deep_link` для обработки команд `/start about`, `/start production` и т.д.), создание веб-лендинга с 6 информационными карточками на HTML/CSS, а также контейнеризацию и развёртывание на сервере. В результате была получена работающая микросервисная архитектура: Telegram-бот и веб-лендинг, защищённый HTTPS с автоматическим обновлением сертификатов. Лендинг содержит кликабельные карточки, при нажатии на которые открывается бот с соответствующей командой. Все сервисы оркестрируются с помощью Docker Compose, а для автоматизации HTTPS используется связка nginx-proxu и acme-companion.

СПИСОК ЛИТЕРАТУРЫ

1. Современная микросервисная архитектура: принципы проектирования. — Текст : электронный // Habr : [сайт]. — URL: <https://habr.com/ru/companies/innotech/articles/683550/> (дата обращения: 30.05.2026)
2. Ньюмен, С. Создание микросервисов / С. Ньюмен. — 2-е изд. — СПб : Питер, 2023. — 624 с. — Текст : непосредственный (дата обращения: 30.05.2026)
3. Ключевые принципы проектирования микросервисов. — Текст : электронный // moluch.ru : [сайт]. — URL: <https://moluch.ru/archive/547/119663> (дата обращения: 30.05.2026).
4. Перальта, Х. А. Микросервисы и API / Х. А. Перальта. — СПб : Питер, 2024. — 464 с. — Текст : непосредственный (дата обращения: 30.05.2026)
5. Монолит или микросервисы: как выбрать архитектуру для нового проекта. — Текст : электронный // Tproger : [сайт]. — URL: <https://tproger.ru/articles/monolit-ili-mikroservisy--kak-vybrat-arhitekturu-dlya-novogo-proekta> (дата обращения: 30.05.2026)
6. Микросервисы vs монолит: разница архитектур и руководство по переходу. — Текст : электронный // work solutions : [сайт]. — URL: <https://worksolutions.ru/blog/mikroservisy-vs-monolit-raznicza-arhitektur-i-rukovodstvo-po-perehodu/> (дата обращения: 30.05.2026)
7. Containers vs Virtual Machines. — Текст : электронный // qsan.io : [сайт]. — URL: <https://qsan.io/ru/blog/containers-vs-virtual-machines/> (дата обращения: 30.05.2026)
8. Архитектура микросервисов Docker. — Текст : электронный // appmaster.io : [сайт]. — URL: <https://appmaster.io/ru/blog/arkhitektura-mikroservisov-docker#docker-и-микросервисы> (дата обращения: 30.05.2026)

9. Микросервис Docker. — Текст : электронный // Purpleschool : [сайт]. — URL: <https://purpleschool.ru/knowledge-base/microservices/infrastructure/mikroservis-docker> (дата обращения: 30.05.2026)
10. Docker. — Текст : электронный // blog.skillfactory.ru : [сайт]. — URL: <https://blog.skillfactory.ru/glossary/docker/> (дата обращения: 30.05.2026)
11. Secure Your Microservices Ingress in Istio with Let's Encrypt. — Текст : электронный // invisibl.io : [сайт]. — URL: <https://invisibl.io/blog/secure-your-microservices-ingress-in-istio-with-lets-encrypt/> (дата обращения: 30.05.2026)
12. SSL-сертификаты от Let's Encrypt с cert-manager в Kubernetes. — Текст : электронный // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/flant/articles/496936/> (дата обращения: 31.05.2026)
13. Let's Encrypt: что это?. — Текст : электронный // mchost.ru : [сайт]. — URL: <https://mchost.ru/articles/lets-encrypt-что-это/> (дата обращения: 31.05.2026)
14. Как работает Let's Encrypt. — Текст : электронный // Let's Encrypt : [сайт]. — URL: <https://letsencrypt.org/ru/how-it-works/> (дата обращения: 31.05.2026)

ПРИЛОЖЕНИЕ А

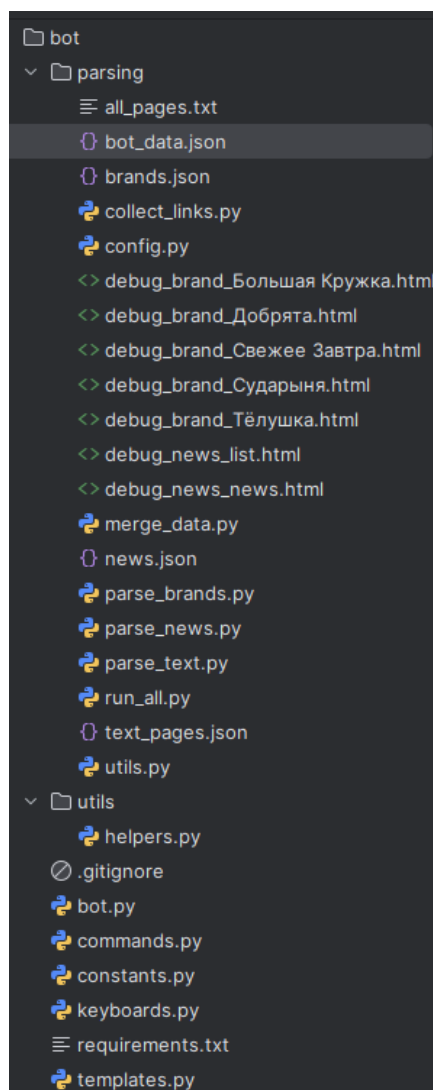


Рисунок 1. Структура Telegram-бота

```
1 {
2   "site_info": {
3     "name": "МногоМолоко",
4     "url": "https://mnogomoloka.ru",
5     "phone": "8 800 555 90 14",
6     "last_updated": "2026-02-19 22:34:07"
7   },
8   "structure": {
9     "sections": {
10      "0 компаний": {
11        "name": "0 компаний",
12        "url": "https://mnogomoloka.ru/about/",
13        "main_page": {
14          "title": "0 компаний",
15          "content": "0 компании Группа компаний «Галактика» – крупный независимый производитель широкого спектра молочных продуктов. «Галактика» полностью реализует всю цепочку",
16          "url": "https://mnogomoloka.ru/about/"
17        },
18        "subsections": {
19          "История": {
20            "title": "История компании",
21            "content": "История компании На сегодняшний день, Группа компаний «Галактика» – один из крупнейших производителей молочных продуктов в России и самый крупный произв",
22            "url": "https://mnogomoloka.ru/history/"
23          },
24          "Миссия и ценности": {
25            "title": "Миссия и ценности",
26            "content": "Миссия и ценности Корпоративная социальная ответственность – одна из важнейших составляющих работы Группы компаний «Галактика». Наша миссия Молоко для л",
27            "url": "https://mnogomoloka.ru/mission/"
28          },
29          "КСО": {
30            "title": "Корпоративная социальная ответственность",
31            "content": "Корпоративная социальная ответственность Корпоративная социальная ответственность – одна из важнейших составляющих работы группы компаний «Галактика».",
32            "url": "https://mnogomoloka.ru/kso/"
33          },
34          "Здоровый образ жизни": {
35            "title": "Здоровый образ жизни",
36            "content": "Здоровый образ жизни «Галактика» поддерживает социальные проекты, связанные с популяризацией здорового питания и здорового образа жизни в целом, включая",
37            "url": "https://mnogomoloka.ru/zog/"
38          },
39        }
40      }
41    }
42  }
```

Рисунок 2. Загрузка данных из parsing/bot_data.json

```
C:\Users\melni\Desktop\BY3\Эксп\Курсовая3\microservice\.venv\Scripts\python.exe C:\Users\melni\Desktop\BY3\Эксп\Курсовая3\microservice\bot\bot.py
2026-06-09 22:36:15,661 - __main__ - INFO - Токен загружен
2026-06-09 22:36:15,662 - __main__ - INFO - Загрузка данных из parsing/bot_data.json...
2026-06-09 22:36:15,673 - commands - INFO - Данные успешно загружены
Данные загружены из parsing/bot_data.json
2026-06-09 22:36:16,256 - __main__ - INFO - Application создан
2026-06-09 22:36:16,256 - __main__ - INFO - Все обработчики зарегистрированы
2026-06-09 22:36:16,256 - __main__ - INFO - Бот запускается...
2026-06-09 22:36:16,740 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8427035340:AAFcyvLrxvpbYjrkH-9dGWzEYdqEsgDx7ME/getMe "HTTP/1.1 200 OK"
2026-06-09 22:36:16,741 - telegram.ext.Application - INFO - Application started
2026-06-09 22:36:16,826 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8427035340:AAFcyvLrxvpbYjrkH-9dGWzEYdqEsgDx7ME/deleteWebhook "HTTP/1.1 200 OK"
2026-06-09 22:36:16,827 - __main__ - INFO - Бот успешно запущен и работает!
```

Рисунок 3. Запуск Telegram-бота

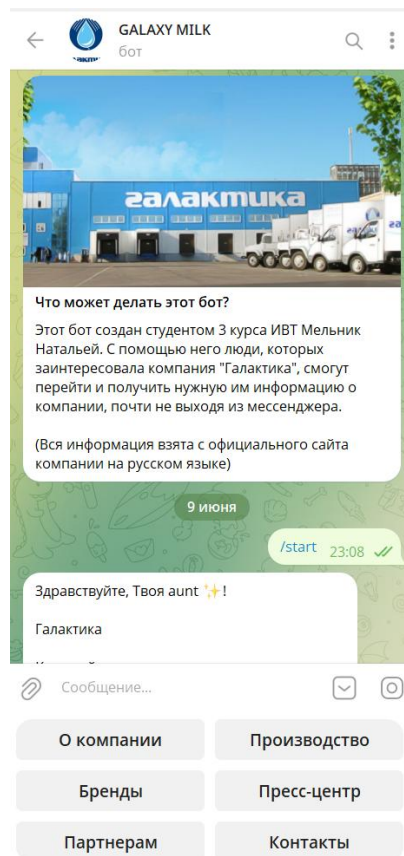


Рисунок 4. Работа Telegram-бота



Рисунок 5. Ссылка на репозиторий в Github

ПРИЛОЖЕНИЕ Б

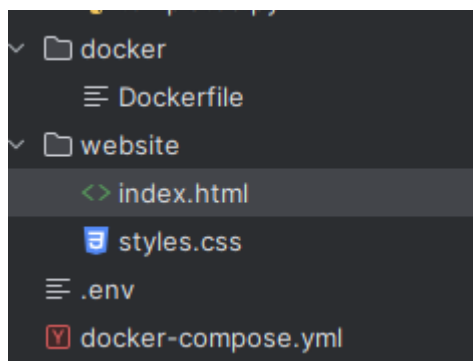


Рисунок 6. Структура веб-лендинга

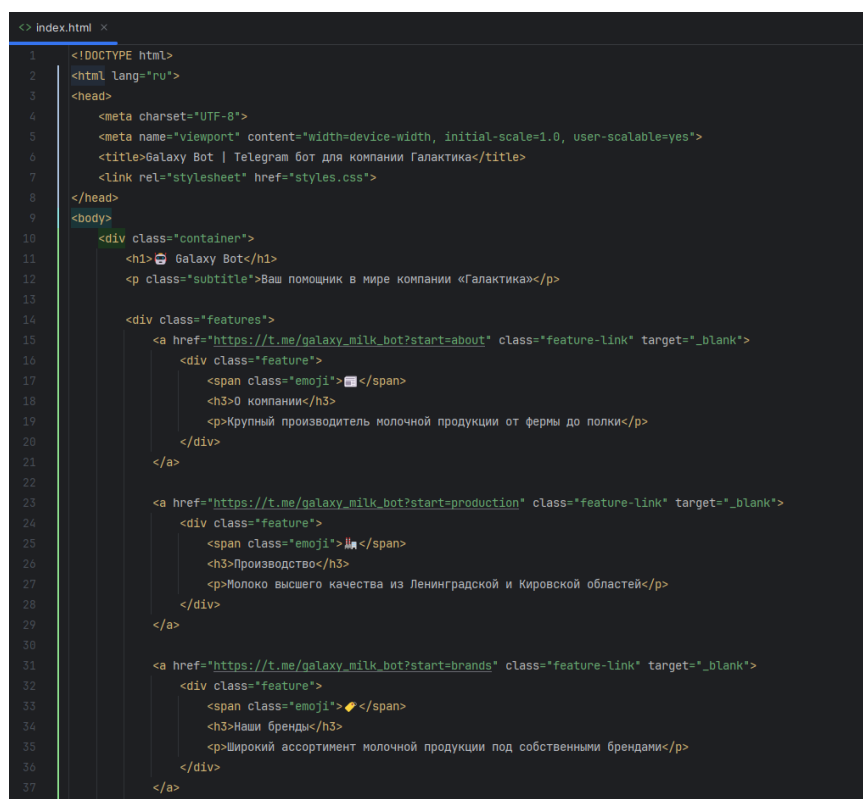


Рисунок 7. HTML-страница веб-лендинга

```
docker-compose.yml  Dockerfile x
Plugins supporting Dockerfile files found.
1 FROM nginx:alpine
2
3 RUN apk add --no-cache curl
4
5 RUN mkdir -p /var/cache/nginx
6
7 COPY nginx/nginx.conf /etc/nginx/nginx.conf
8
9 EXPOSE 80
10
11 CMD ["nginx", "-g", "daemon off;"]
```

Рисунок 8. Dockerfile

```
docker-compose.yml x
1 version: '3.8'
2
3 networks:
4   webproxy:
5     driver: bridge
6
7 services:
8   # 1. Nginx Proxy (c Let's Encrypt)
9   nginx-proxy:
10    image: nginxproxy/nginx-proxy:latest
11    container_name: nginx-proxy
12    restart: unless-stopped
13    ports:
14      - "80:80"
15      - "443:443"
16    volumes:
17      - /var/run/docker.sock:/tmp/docker.sock:ro
18      - ./nginx/conf.d:/etc/nginx/conf.d
19      - ./nginx/vhost.d:/etc/nginx/vhost.d
20      - ./nginx/html:/usr/share/nginx/html
21      - ./nginx/certs:/etc/nginx/certs:ro
22    networks:
23      - webproxy
24    labels:
25      - "com.github.jrcs.letsencrypt_nginx_proxy_companion.nginx_proxy=true"
26
27   # 2. Помощник для сертификатов
28   letsencrypt-companion:
29    image: nginxproxy/acme-companion:latest
30    container_name: nginx-letsencrypt
31    restart: unless-stopped
32    volumes:
33      - /var/run/docker.sock:/var/run/docker.sock:ro
34      - ./nginx/certs:/etc/nginx/certs:rw
35      - ./nginx/vhost.d:/etc/nginx/vhost.d
36      - ./nginx/html:/usr/share/nginx/html
37    environment:
38      - DEFAULT_EMAIL=me@lnik3570@gmail.com
```

Рисунок 9. Файл docker-compose.yml

```
PS C:\Users\melni> scp -r C:\Users\melni\Desktop\BV3\3курс\Курсовая3\microservice root@31.184.254.21:/root/
.env 100% 56 3.2KB/s 00:00
.gitignore 100% 50 2.0KB/s 00:00
profiles_settings.xml 100% 174 9.4KB/s 00:00
Project_Default.xml 100% 671 25.2KB/s 00:00
microservice.iml 100% 362 20.8KB/s 00:00
misc.xml 100% 314 18.0KB/s 00:00
modules.xml 100% 283 15.4KB/s 00:00
workspace.xml 100% 2365 32.5KB/s 00:00
.gitignore 100% 42 2.1KB/s 00:00
81d243bd2c585b0f4821__mypyc.cp312-win_amd64.pyd 100% 214KB 599.4KB/s 00:00
hdrs.py.hash 100% 64 3.5KB/s 00:00
_cparser.pxd.hash 100% 64 3.7KB/s 00:00
_find_header.pxd.hash 100% 64 4.2KB/s 00:00
_helpers.pyi.hash 100% 64 3.5KB/s 00:00
_helpers.pyx.hash 100% 64 3.5KB/s 00:00
_http_parser.pyx.hash 100% 64 4.2KB/s 00:00
_http_writer.pyx.hash 100% 64 2.5KB/s 00:00
_websocket.pyx.hash 100% 64 3.7KB/s 00:00
abc.py 100% 5749 233.9KB/s 00:00
base_protocol.py 100% 2831 115.2KB/s 00:00
client.py 100% 47KB 596.7KB/s 00:00
client_exceptions.py 100% 9791 308.4KB/s 00:00
client_proto.py 100% 8888 434.0KB/s 00:00
```

Рисунок 10. Копирование файлов проекта на сервер

```
root@vdsplit:~/microservice# mkdir -p nginx/{conf.d,vhost.d,html,certs}
root@vdsplit:~/microservice# docker --version
Docker version 29.5.2, build 79eb04c
root@vdsplit:~/microservice# docker-compose --version
Docker Compose version v5.1.4
```

Рисунок 11. Создание папок nginx и проверка версий Docker

```
root@vdsplit:~/microservice# docker-compose up -d
WARN[0000] /root/microservice/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove
it to avoid potential confusion
[+] up 4/4
✔ Network microservice_webproxy Created 0.2s
✔ Container nginx-proxy Started 2.5s
✔ Container nginx-letsencrypt Started 2.6s
✔ Container website Started 2.4s
```

Рисунок 12. Запуск контейнеров

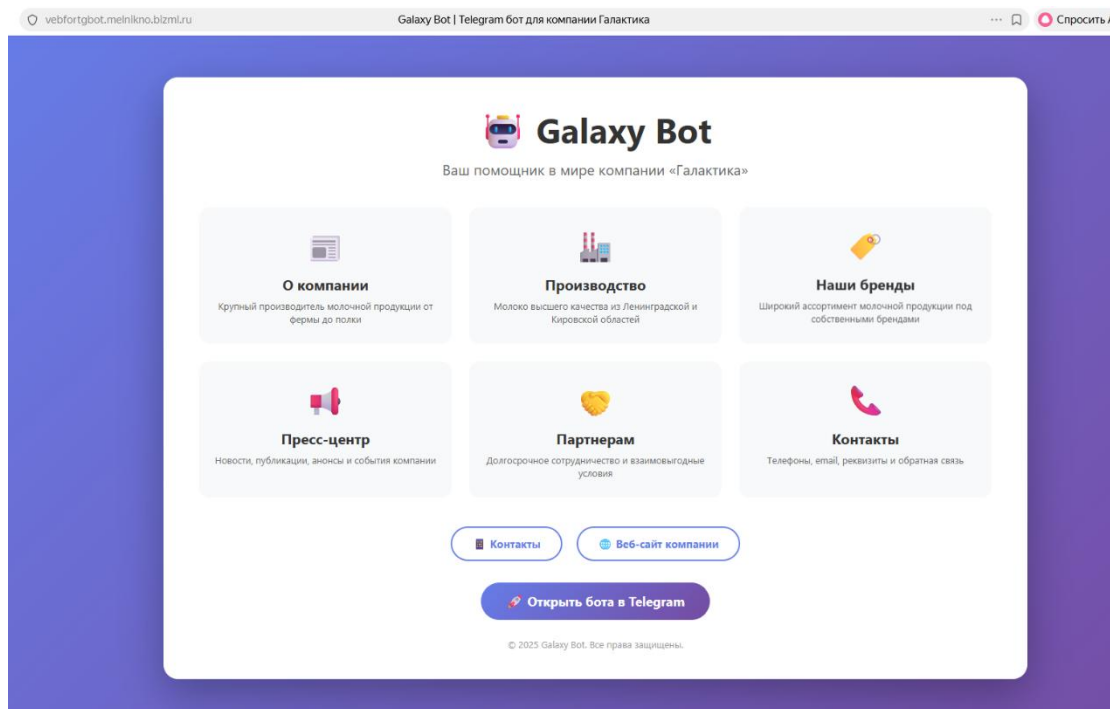


Рисунок 13. Созданный веб-лендинг



Рисунок 14. Ссылка на веб-лендинг



Рисунок 15. Ссылка на репозиторий в Github (Telegram-бот + веб-лендинг)